1] Instance Variable - Variables defined inside the init method using self keyword is known as instance variable.

eg.

```
Class Employee :
    def __init__ (self, emp_name, dep):
        self.emp_name = emp_name
        self.dep = dep

empl = Employee ("rahul", "IT")

print (empl.emp_name)  → rahul
print (empl.dep)       → IT
```

empl → empl, rahul → emp_name, IT → dep

emp_name = emp_name, dep = dep } instance variable

2] Static Variables:-

Static variables are inside the class

Shared among all objects.

Instance variables	Static variables
① inside --init-- method.	① inside class
② for each object it has Unique own value	② shared same value across the object
③ obj_name.inst_var_name	③ class_name.stat_var_name
④ eg. user_name acc_bal	④ eg. ifsc_code address

Instance method	Static method
① first parameter - self	① no self.
② access to instance variable - yes (obj_name.ins.varnam)	② access to instance variable - NO (No object reference)
③ access to static	③ access to static

Variable - yes
class-name.stat-var name

④ How to access -
obj.name.method()

⑤ Use case - act on specific obj's data.

④ How to access -
Class-Name.method()

⑤ Use case -
Utility/helper function, class level data.

Static method -

Static method is an utility function, Conceptually belongs to class, but cannot access specific data of an object.

Instance variable - Not able to access

- mention @Staticmethod above def keyword

When to use Static method.

① helper/utility function - method is conceptually belong to class

but does not require specific object data.

Instance method -
first parameter is self keyword. can access instance variable & static variable.

Encapsulation -
Data hiding + controlled access

eg. class ATM:
    def __init__(self):
        self.secret_pin = ""
        self.account_bal = 5000

```
def set_pin(self):
    self.secret_pin = input("enter
                        the pin")

def view_bal(self):
    print("your balance is",
          self.account_bal)

obj1 = ATM()
obj1.account_bal = 0
```

Problem - Python allows anyone to modify object attributes directly from outside the class.

- Using Encapsulation we make those variables private variable.

→ __prefix

```
self.account_bal = 5000
```



self.__account_balance = 5000